ЛАБОРАТОРНАЯ РАБОТА 1

МЕТОДЫ ПАКЕТНОЙ ОБРАБОТКИ ДАННЫХ

1. Написать и запустить классический MapReduce «WordCount» на HDFS

а. Подготовка input-файла (100 MB текста)

Для выполнения задачи был написан Python-скрипт `generate.py`, генерирующий тестовый файл из повторяющихся строк слов общим объемом около 100 МБ. Размер сгенерированного файла `input.txt` составил ~100 МБ.



б. Компиляция и запуск Java-приложения

Был написан классический код MapReduce на Java (`WordCount.java`). В коде реализованы:

- **TokenizerMapper**: разбивает входные строки на слова и генерирует пары `(слово, 1)`.
- **IntSumReducer**: суммирует значения для каждого ключа.
- **Combiner**: выполняет локальное суммирование на стороне Mapper для уменьшения объема данных, передаваемых по сети.

Исходный файл был скопирован в HDFS:

```
hdfs dfs -mkdir -p /input
hdfs dfs -put /input.txt /input/
```

Программа была скомпилирована с использованием классов Hadoop, упакована в JAR-архив и отправлена на выполнение в кластер:

```
mkdir classes
javac -classpath $(hadoop classpath) -d classes WordCount.java
jar -cf wordcount.jar -C classes .
hadoop jar wordcount.jar WordCount /input /output
```

```
2026-09-29 12:21:54,834 INFO client.RMProxy: Connecting to ResourceManager at resourcemanager/172.19.0.3:8032
2026-09-29 12:21:55,093 INFO client.AHSProxy: Connecting to Application History server at historyserver/172.19.0.2:10200
2026-09-29 12:21:55,397 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool int
erface and execute your application with ToolRunner to remedy this.
2026-09-29 12:21:55,431 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/root/.stagin
g/job_1790683595108_0001
2026-09-29 12:21:55,628 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrusted =
 false
2026-09-29 12:21:56,246 INFO input.FileInputFormat: Total input files to process : 1
2026-09-29 12:21:56,381 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrusted =
 false
2026-09-29 12:21:56,412 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrusted =
 false
2026-09-29 12:21:56,828 INFO mapreduce.JobSubmitter: number of splits:1
2026-09-29 12:21:56,981 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrusted =
 false
2026-09-29 12:21:57,408 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1790683595108_0001
2026-09-29 12:21:57,408 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-29 12:21:57,624 INFO conf.Configuration: resource-types.xml not found
2026-09-29 12:21:57,625 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-29 12:21:58,132 INFO impl.YarnClientImpl: Submitted application application_1790683595108_0001
2026-09-29 12:21:58,185 INFO mapreduce.Job: The url to track the job: http://resourcemanager:8088/proxy/application_1790683595108_00
01/
2026-09-29 12:21:58,186 INFO mapreduce.Job: Running job: job_1790683595108_0001
2026-09-29 12:22:08,484 INFO mapreduce.Job: Job job_1790683595108_0001 running in uber mode : false
2026-09-29 12:22:08,485 INFO mapreduce.Job:  map 0% reduce 0%
```

в. Анализ вывода

Результат работы задачи был выгружен из HDFS командой `hdfs dfs -cat /output/part-r-00000`:

```
root@eeb9d919ed5d:/# hdfs dfs -cat /output/part-r-00000
2026-09-29 12:23:51,609 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrusted =
 false
Daniil  1700000
Ilyinchyk       1700000
big     1700000
data    1700000
hadoop  1700000
hello   3400000
spark   1700000
world   3400000
root@eeb9d919ed5d:/#
```

Аналитический разбор метрик Map-Reduce Framework:

1. **Сплиты и параллелизм:**
   - `Launched map tasks = 1`
   - `Launched reduce tasks = 1` Файл размером 100 МБ меньше стандартного размера блока HDFS (128 МБ), поэтому он целиком лег в один блок (Input Split) и был обработан одним маппером.
2. **Объем входных и выходных записей Mapper:**
   - `Map input records = 1 700 000` (маппер прочитал ровно 1.7 млн строк).
   - `Map output records = 17 000 000` (каждая строка содержала по 10 слов, итого сгенерировано 17 млн пар вида `(слово, 1)`).
3. **Эффективность Combiner:**
   - Без оптимизации объем данных на отправку через сеть составил бы **175 100 000 байт** (`Map output bytes`).
   - За счет включения `Combiner` предварительное суммирование произошло локально на ноде прямо в оперативной памяти до передачи по сети.
   - Количество переданных записей упало с 17 млн до **56** (`Combine output records`).

- По сети на стадию Reduce было передано всего **96 байт** (`Reduce shuffle bytes`). Это демонстрирует критическую важность использования Combiner для оптимизации сетевой нагрузки (Shuffle).

4. **Результат на стадии Reduce:**
   - `Reduce input groups = 8` — было выделено 8 уникальных слов.
   - `Reduce output records = 8` — на выходе сформирован файл с подсчетом вхождений по каждому слову.

---

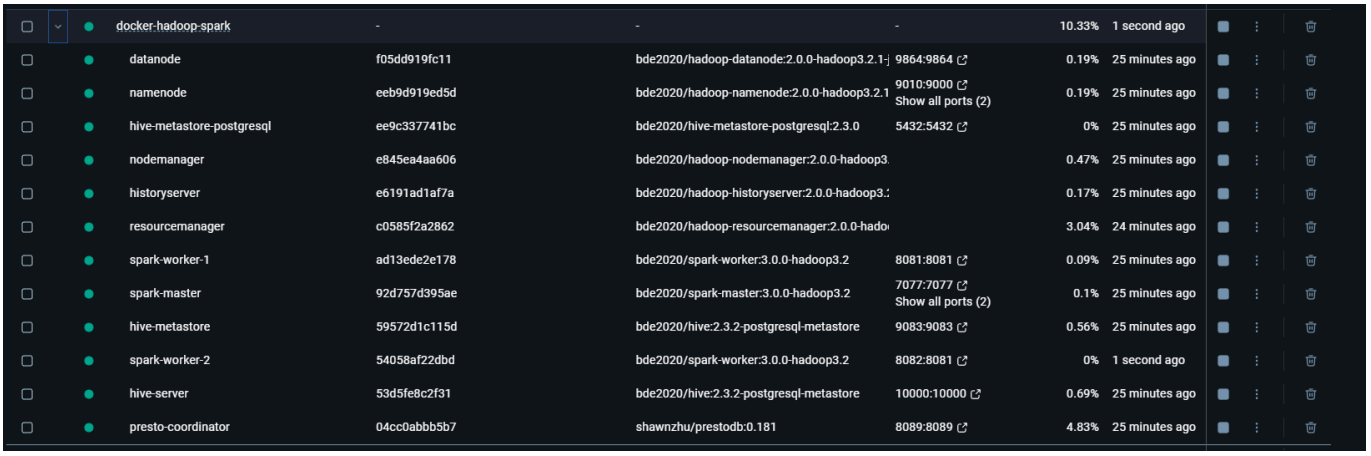
2. Развернуть кластер Spark (master + 2 workers) в Docker

Для выполнения задач обработки данных был развернут кластер Apache Spark версии 3.0.0 на базе Docker Compose в конфигурации **1 Master + 2 Worker**.

В конфигурацию `docker-compose.yml` был добавлен второй воркер со следующими параметрами:

```
  spark-worker-2:
    image: bde2020/spark-worker:3.0.0-hadoop3.2
    container_name: spark-worker-2
    depends_on:
      - namenode
      - datanode
      - spark-master
    ports:
      - "8082:8081"
    environment:
      - "CORE_CONF_fs_defaultFS=hdfs://namenode:9000"
      - "SPARK_MASTER=spark://spark-master:7077"
      - "SPARK_WORKER_CORES=1"
      - "SPARK_WORKER_MEMORY=1G"
      - "SPARK_WORKER_PORT=8881"
      - "SPARK_WORKER_WEBUI_PORT=8081"
```

Кластер успешно запущен, оба воркера зарегистрированы на мастере.



docker-hadoop-spark	-	-	-	10.33%	1 second ago
datanode	f05dd919fc11	bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-j	9864:9864	0.19%	25 minutes ago
namenode	eeb9d919ed5d	bde2020/hadoop-namenode:2.0.0-hadoop3.2.1	9010:9000 Show all ports (2)	0.19%	25 minutes ago
hive-metastore-postgresql	ee9c337741bc	bde2020/hive-metastore-postgresql:2.3.0	5432:5432	0%	25 minutes ago
nodemanager	e845ea4aa606	bde2020/hadoop-nodemanager:2.0.0-hadoop3.		0.47%	25 minutes ago
historyserver	e6191ad1af7a	bde2020/hadoop-historyserver:2.0.0-hadoop3.2		0.17%	25 minutes ago
resourcemanager	c0585f2a2862	bde2020/hadoop-resourcemanager:2.0.0-hado		3.04%	24 minutes ago
spark-worker-1	ad13ede2e178	bde2020/spark-worker:3.0.0-hadoop3.2	8081:8081	0.09%	25 minutes ago
spark-master	92d757d395ae	bde2020/spark-master:3.0.0-hadoop3.2	7077:7077 Show all ports (2)	0.1%	25 minutes ago
hive-metastore	59572d1c115d	bde2020/hive:2.3.2-postgresql-metastore	9083:9083	0.56%	25 minutes ago
spark-worker-2	54058af22dbd	bde2020/spark-worker:3.0.0-hadoop3.2	8082:8081	0%	1 second ago
hive-server	53d5fe8c2f31	bde2020/hive:2.3.2-postgresql-metastore	10000:10000	0.69%	25 minutes ago
presto-coordinator	04cc0abbb5b7	shawnzhu/prestodb:0.181	8089:8089	4.83%	25 minutes ago

---

3. Написать Spark-приложение на Python/Scala

a. Реализация WordCount (RDD и DataFrame API)

Для реализации подсчета слов были разработаны два приложения на Python с использованием библиотеки PySpark. Так как вычисления выполняются распределенно на нескольких воркерах кластера, входные данные считываются напрямую из общего распределенного хранилища HDFS (`hdfs://namenode:9000/input/input.txt`), и результаты сохраняются туда же.

---

1. Реализация через Spark RDD API (`wordcount_rdd.py`)

Низкоуровневый API, оперирующий распределенными коллекциями объектов (Resilient Distributed Datasets).

- `textFile` — ленивое чтение файла построчно.
- `flatMap` — разделение каждой строки на слова по пробельным символам.
- `map` — преобразование каждого слова в кортеж вида `(слово, 1)`.
- `reduceByKey` — агрегация и суммирование счетчиков для одинаковых слов.

Запуск задачи на кластере:

```
/spark/bin/spark-submit --master spark://spark-master:7077 \
  wordcount_rdd.py \
  hdfs://namenode:9000/input/input.txt \
  hdfs://namenode:9000/output_rdd
```

Результат выполнения WordCount на базе RDD:

```
PS D:\VSCode\Python\BSU\AODB> docker exec namenode hdfs dfs -cat /output_rdd/part-*
2026-09-29 12:57:36,711 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrust
ed = false
('world', 3400000)
('hadoop', 1700000)
('Ilyinchyk', 1700000)
('hello', 3400000)
('spark', 1700000)
('big', 1700000)
('data', 1700000)
('Daniil', 1700000)
```

---

2. Реализация через Spark DataFrame API (`wordcount_df.py`)

Высокоуровневый декларативный API, представляющий данные в виде структурированной таблицы со схемой. Под капотом Spark строит оптимизированный логический и физический план выполнения с помощью Catalyst Optimizer.

- `spark.read.text` — считывание данных в DataFrame с колонкой `value`.
- `split` и `explode` — разбиение текста и разворачивание массива в отдельные строки (одно слово на строку).
- `groupBy("word").count()` — декларативная группировка и подсчет, аналогичная SQL-запросу.

Запуск задачи на кластере:

```
/spark/bin/spark-submit --master spark://spark-master:7077 \
  wordcount_df.py \
  hdfs://namenode:9000/input/input.txt \
  hdfs://namenode:9000/output_df
```

Результат выполнения WordCount на базе DataFrame:

```
PS D:\VSCode\Python\BSU\AODB> docker exec namenode hdfs dfs -cat /output_df/part-*
2026-09-29 12:57:54,014 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrust
ed = false
Daniil,1700000
hello,3400000
Ilyinchyk,1700000
data,1700000
spark,1700000
world,3400000
hadoop,1700000
big,1700000
PS D:\VSCode\Python\BSU\AODB>
```

Вывод по пункту 3.а: Оба подхода дали идентичные результаты подсчета, полностью совпадающие с классическим Hadoop MapReduce (1 700 000 вхождений для каждого слова). При этом разработка на Spark DataFrame API является наиболее лаконичной, читаемой и производительной за счет автоматических внутренних оптимизаций Spark SQL.

б. Реализация алгоритма PageRank на графе из 1000 вершин

Алгоритм PageRank моделирует поведение пользователя («случайного серфера»), переходящего по гиперссылкам в графе.

Был сгенерирован граф из 1000 вершин (от 0 до 999) и ~6000 направленных ребер в формате `src \t dst`, сохраненный в файл `graph_edges.txt` и загруженный в HDFS.

Особенности реализации:

1. Вершины без исходящих ребер (тупики) инициализируются пустыми списками ребер через объединение с полным множеством вершин (`sc.parallelize(range(num_vertices))`).
2. Граф ссылок `links` кэшируется в оперативной памяти с помощью `.persist()`, так как структура графа неизменна и используется повторно на каждой итерации.
3. Используется классическая формула затухания с коэффициентом $d = 0.85$:

$$Rank(v) = 0.15 + 0.85 \times \sum \frac{Rank(u)}{Count(u)}$$

4. Расчет выполнен за 10 итераций.

Запуск задачи на кластере:

```
/spark/bin/spark-submit --master spark://spark-master:7077 \
  pagerank.py \
  hdfs://namenode:9000/graph_edges.txt \
  hdfs://namenode:9000/output_pagerank 1000 10
```

Результаты работы алгоритма (топ-10 вершин с максимальным рангом):

```
PS D:\VSCode\Python\BSU\AODB> docker exec namenode bash -c "hdfs dfs -cat /output_pagerank/part-* | head -10"
2026-09-29 13:08:56,264 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrust
ed = false
(3, 23.166686287557717)
(7, 20.306247765877153)
(15, 19.498337590488738)
(0, 18.452671708210413)
(11, 17.302680833391836)
(6, 16.54839064026855)
(2, 16.42402963160023)
(14, 16.32053624361112)
(1, 15.055028614848707)
(5, 14.924327211664757)
```

Проверка общего количества рассчитанных вершин (ровно 1000):

```
PS D:\VSCode\Python\BSU\AODB> docker exec namenode bash -c "hdfs dfs -cat /output_pagerank/part-* | wc -l"
2026-09-29 13:09:12,226 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrust
ed = false
1000
PS D:\VSCode\Python\BSU\AODB>
```

---

4. Исследовать влияние партиционирования

Spark делит распределенные наборы данных на логические фрагменты — партиции. Каждая партиция обрабатывается отдельной задачей (Task) в одном потоке ядра процессора.

- Если число партиций меньше количества доступных процессорных ядер кластера — ядра простаивают, параллелизм не используется на максимум.
- Если число партиций слишком велико — выигрыш от распараллеливания нивелируется накладными расходами на планирование задач, сериализацию, сетевой обмен и мониторинг.

Был проведен эксперимент замера времени выполнения WordCount при разном количестве партиций:

```
RESULT: partitions=1 actual=1 time_ms=26154
RESULT: partitions=2 actual=2 time_ms=13691
RESULT: partitions=4 actual=4 time_ms=8965
RESULT: partitions=8 actual=8 time_ms=10018
RESULT: partitions=16 actual=16 time_ms=10678
RESULT: partitions=32 actual=32 time_ms=10271
RESULT: partitions=128 actual=128 time_ms=17412
bash-5.0#
```

Анализ результатов:

1. При `partitions=1` время составило **26 154 мс** (вычисления шли строго в один поток).
2. При увеличении до `partitions=2` время сократилось почти в 2 раза — до **13 691 мс**.
3. **Оптимум был достигнут на `partitions=4`** с временем **8 965 мс** — это наилучший результат, так как число задач совпало с суммарным числом доступных вычислительных ядер кластера.
4. При дальнейшем дроблении данных время снова начало возрастать: 8 партиций — **10 018 мс**, 32 партиции — **10 271 мс**, а при 128 партициях время выросло до **17 412 мс**. Накладные расходы на

управление 128 микрозадачами на файле в 100 МБ замедлили работу почти в 2 раза по сравнению с оптимумом.

---

5. Использовать repartition / coalesce

Операции `repartition` и `coalesce` служат для изменения числа партиций уже существующего RDD в памяти, но работают принципиально по-разному:

- `repartition(n)` выполняет полное перемешивание данных по сети между узлами (**Full Shuffle**). Позволяет как увеличивать, так и уменьшать число партиций, распределяя данные максимально равномерно.
- `coalesce(n)` оптимизирован для объединения партиций внутри одного узла без сетевого Shuffle (Narrow Dependency). Может только уменьшать число партиций.

Было выполнено сравнение обеих функций на исходном RDD с базовым числом партиций `actual=2`:

```
RESULT: base actual=2 time_ms=12719
RESULT: repartition(1) actual=1 time_ms=22510
RESULT: coalesce(1) actual=1 time_ms=21836
RESULT: repartition(2) actual=2 time_ms=14886
RESULT: coalesce(2) actual=2 time_ms=11700
RESULT: repartition(4) actual=4 time_ms=9795
RESULT: coalesce(4) actual=2 time_ms=14067
RESULT: repartition(8) actual=8 time_ms=11837
RESULT: coalesce(8) actual=2 time_ms=11542
RESULT: repartition(16) actual=16 time_ms=10448
RESULT: coalesce(16) actual=2 time_ms=12194
bash-5.0#
```

Анализ результатов:

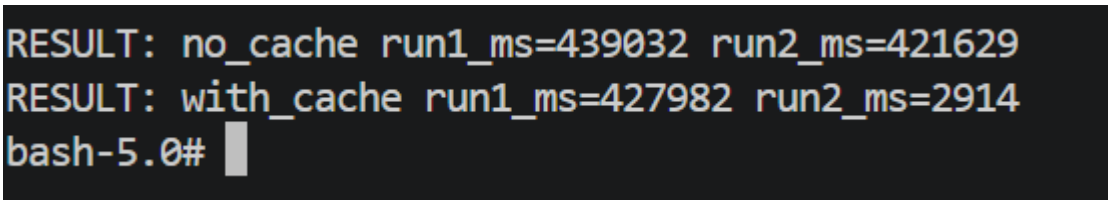
1. **Поведение `coalesce` при попытке увеличения:** При запросе на увеличение числа партиций (`coalesce(4)`, `coalesce(8)`, `coalesce(16)`) фактическое число партиций осталось равным **2** (`actual=2`). `coalesce` не инициирует shuffle и не умеет дробить существующие партиции на большее число.
2. **Поведение `repartition`:** Легко увеличил число партиций до целевых 4, 8 и 16 за счет выполнения полного Shuffle.
3. **Производительность при уменьшении (цель = 1):** `coalesce(1)` отработал за **21 836 мс**, а `repartition(1)` — за **22 510 мс**. `coalesce` работает быстрее, поскольку ему не требуется дорогостоящее межсетевое перераспределение данных.

---

6. Использование persist / cache

Spark использует парадигму «ленивых вычислений» (Lazy Evaluation). По умолчанию Spark пересчитывает весь граф преобразований (Lineage) с самого начала (с чтения файла) при каждом вызове терминального действия (Action, например `.count()`). Методы `.cache()` / `.persist()` позволяют зафиксировать вычисленный RDD в оперативной памяти для мгновенного переиспользования.

В ходе эксперимента было выполнено два последовательных действия `.count()` над RDD с тяжелой вычислительной функцией — сначала без кэширования, затем с кэшированием через `.persist()`:



Анализ результатов:

1. **Без кэширования (`no_cache`):**
    - Запуск 1: **439 032 мс** (~7.3 минут).
    - Запуск 2: **421 629 мс** (~7.0 минут).
    - Оба раза Spark полностью заново считывал файл и повторно выполнял миллионы тяжелых операций хеширования.
2. **С кэшированием (`with_cache`):**
    - Запуск 1: **427 982 мс** (Spark выполнил расчет и сохранил результат в RAM воркеров).
    - Запуск 2: **2 914 мс** (менее 3 секунд!).
3. **Вывод:** Кэширование RDD дало колоссальное ускорение во втором запуске — более чем в 145 раз. Это подтверждает ключевое преимущество Spark при итеративных алгоритмах (например, PageRank или обучение ML-моделей).

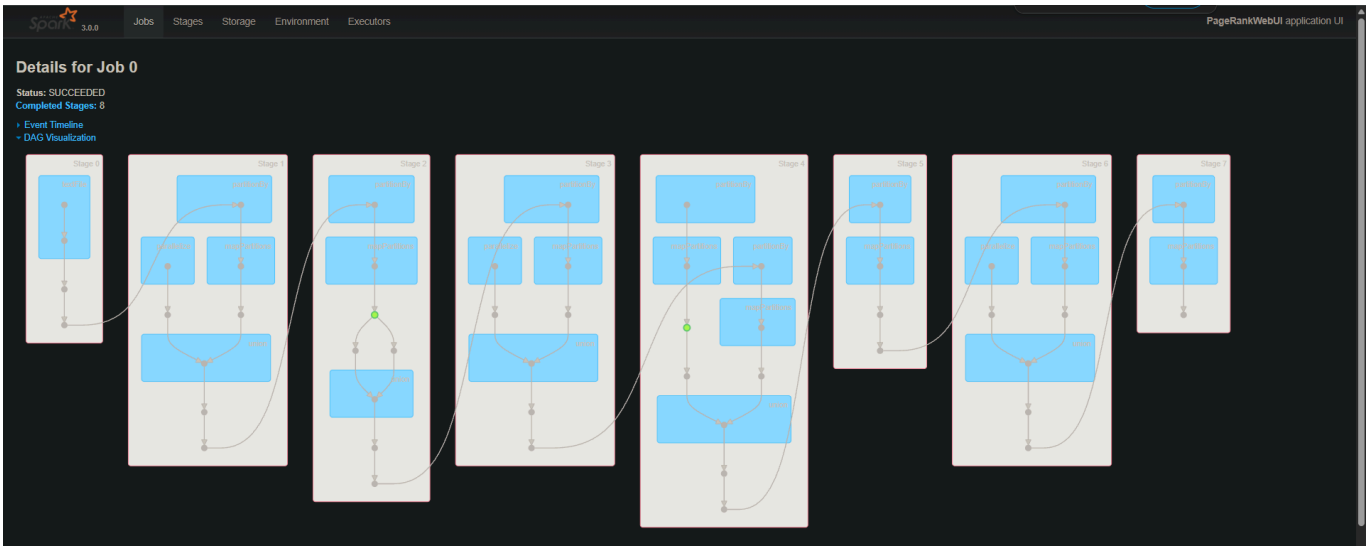
---

7. Использовать Spark Web UI

Для мониторинга, отладки и визуализации работы распределенного приложения был использован встроенный веб-интерфейс **Spark Web UI** (порт `4040`).

а. Анализ графа выполнения, стадий и задач

Spark компилирует ленивые RDD-трансформации в направленный ациклический граф (DAG — Directed Acyclic Graph), объединяя независимые операции и разделяя вычисления на стадии:

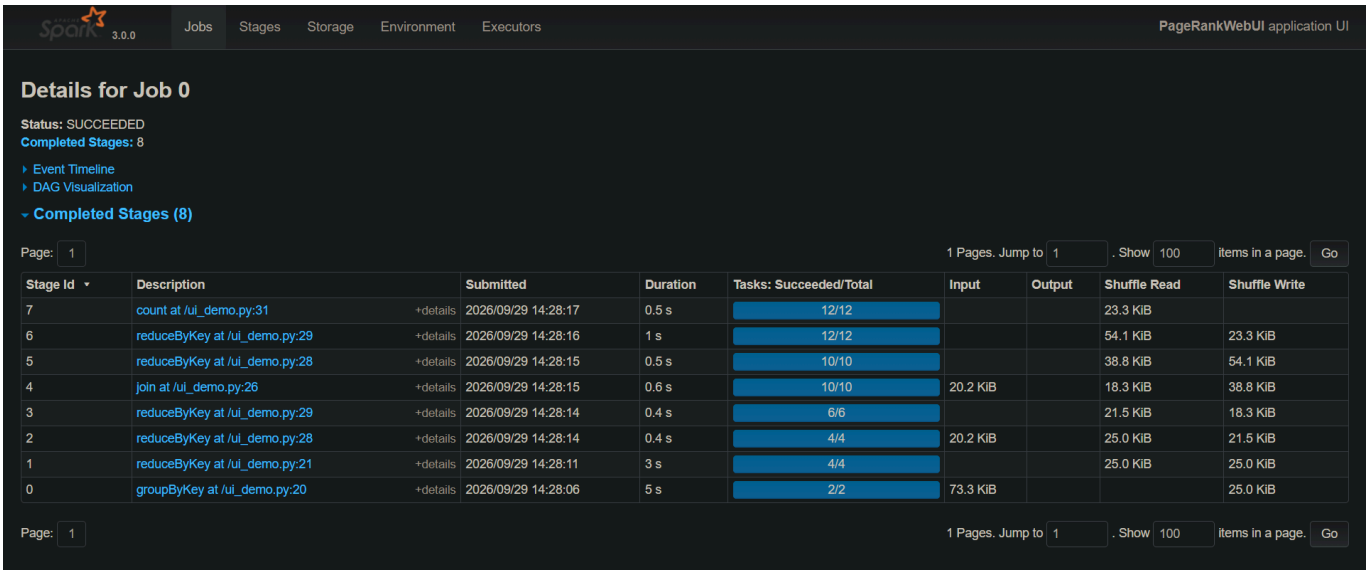


1. **Разбиение на стадии (Stages):**
   - Граф разбит на 8 стадий (Stage 0 — Stage 7). Границами между стадиями (серые соединительные стрелки между большими блоками) являются операции, требующие **Shuffle** (перемешивания данных по сети между узлами): `groupByKey`, `reduceByKey`, `join`.
   - Внутри каждой стадии операции выполняются в конвейерном режиме (**Pipelining**): например, цепочки `mapPartitions` и `union` выполняются последовательно в памяти без сброса на диск и без передачи по сети.
2. **Кэширование на графе DAG:**
   - На схеме в блоках Stage 2 и Stage 4 четко видны **зеленые точки**. Это индикатор того, что данный RDD был зафиксирован в памяти с помощью `.persist()`. Spark не пересчитывает граф заново от источника, а мгновенно подтягивает состояние RDD из оперативной памяти.

---

6. Анализ Shuffle-операций и хранилища

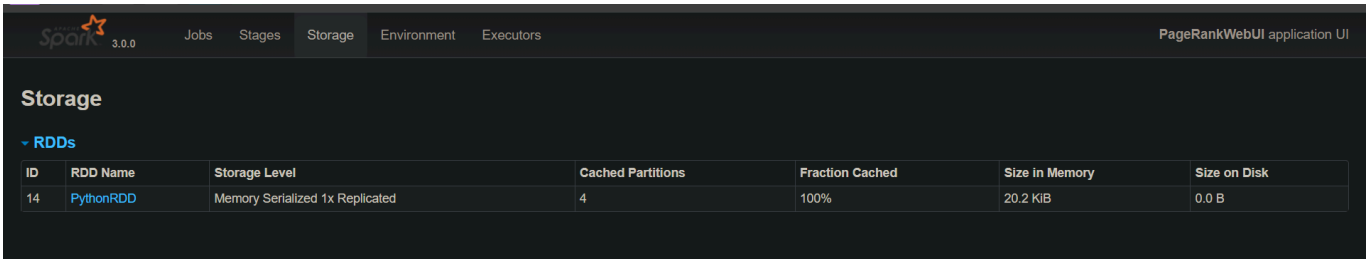
Метрики передачи данных между стадиями подробно представлены в таблице выполненных этапов:



- **Shuffle Read / Write:** В столбцах `Shuffle Read` и `Shuffle Write` зафиксирован реальный межсетевой трафик. Например, `groupByKey` на Stage 0 сформировал `Shuffle Write = 25.0 KiB`, который затем был передан по сети и вычитан на Stage 1 (`Shuffle Read = 25.0 KiB`).

- **Задачи:** Число задач на каждой стадии строго соответствует числу обрабатываемых партиций (например, 2/2 на Stage 0, 4/4 на Stage 1 и 2, до 12/12 на финальных стадиях).

Состояние оперативной памяти воркеров отображается во вкладке **Storage**:



- RDD графа связей (`links`) зафиксирован со статусом `Memory Serialized 1x Replicated`.
- `Fraction Cached: 100%` — все 4 партиции полностью поместились в RAM (размер в памяти `20.2 KiB`), обращений к диску не потребовалось (`Size on Disk = 0.0 B`).

---

8. Сравнение подходов Hadoop MapReduce и Apache Spark

На основе выполненных практических заданий проведено сравнение двух вычислительных парадигм:

Критерий	Apache Hadoop MapReduce	Apache Spark
Архитектура хранения	Запись промежуточных данных на диск (HDFS) после каждой стадии (Map и Reduce).	Хранение данных в оперативной памяти (In-Memory Processing). Сброс на диск происходит только при нехватке RAM.
Производительность	Медленная пакетная обработка из-за постоянных дисковых и сетевых операций ввода-вывода.	В десятки раз быстрее (в эксперименте повторный запуск с кэшем отработал за 2.9 с против 421 с без кэша — ускорение в ~145 раз).
Удобство разработки	Низкое. Требует написания большого объема шаблонного Java-кода, ручной сериализации (типы `Text`, `IntWritable`) и жесткой привязки к схеме Map -> Reduce.	Высокое. Лаконичный API (Python, Scala, SQL). Наличие оптимизатора запросов Catalyst в DataFrame API. Код в разы компактнее и понятнее.
Повторное использование данных	Крайне неэффективно. Итеративные алгоритмы (PageRank, графы, ML) вынуждены перезаписывать и считывать промежуточные данные с диска на каждом шаге цикла.	Идеально. Методы `.cache()` и `.persist()` позволяют один раз зафиксировать RDD в RAM и переиспользовать его в цикле без дополнительных затрат.

Итоговый вывод

Apache Spark превосходит классический Hadoop MapReduce по всем ключевым параметрам: скорости вычислений за счет In-Memory архитектуры, гибкости API и эффективности работы с итеративными алгоритмами. При этом экосистема Hadoop (HDFS) сохраняет свою актуальность как надежное распределенное хранилище первичных сырых данных, поверх которого работает вычислительный движок Spark.